Lab 03 — Sistema Académico + Refactor por módulos con Ports

Integrantes

Jhonnatan David Hernandez Martinez - 2201492
Diego Alejandro Gómez Pedraza - 2211060
Luis Sebastián Mora Cañas - 2211554

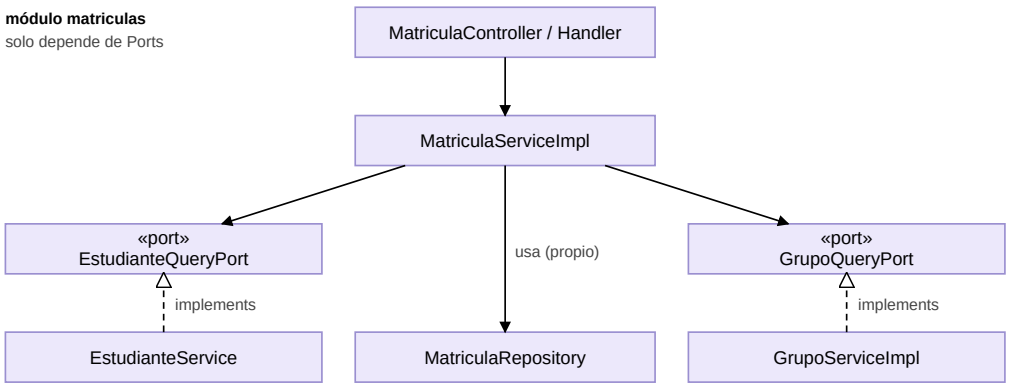
Se construyó el flujo Programa → Asignatura → Grupo → Matrícula (Parte 1) y luego se reorganizó el proyecto por módulos, con Matrícula comunicándose con Estudiantes y Grupos solo mediante Ports (Parte 2).

Actividades

1. Se crearon las entidades `Programa`, `Asignatura`, `Grupo` y `Matricula`, con sus DTOs, repositorios, services, handlers y controllers.
2. Como el Lab 02 se hizo con la opción B, se agregaron las piezas de la opción A que pide este lab: `ApiResponse`, `ResponseBuilder`, `BusinessException` (409) y `NotFoundException` (404). Las excepciones del Lab 02 ahora heredan de ellas.
3. Se implementaron las reglas de matrícula: no duplicado, no superar el cupo y existencia del estudiante y del grupo.
4. Se creó `DataSeederAcademico` (3 programas, 10 asignaturas, 20 grupos) y se bajó el seeder de estudiantes a 50 (`app.seed.cantidad-estudiantes`).
5. Se movieron las clases a los paquetes `common`, `estudiantes`, `programas`, `asignaturas`, `grupos` y `matriculas`, sin cambiar los endpoints.
6. Se crearon `EstudianteQueryPort` y `GrupoQueryPort`; `EstudianteService` y `GrupoServiceImpl` los implementan.
7. `MatriculaServiceImpl` dejó de inyectar `EstudianteRepository` y `GrupoRepository` y ahora usa solo los Ports.
8. Se agregó la prueba `ArquitecturaMatriculasTest`, que falla si el módulo matriculas importa un repositorio de otro módulo, y un README corto.
9. Se ejecutó el mismo guion de pruebas antes y después del refactor.

Nota: el proyecto corre en el puerto 8081 (`application.yaml`), así que Swagger está en `http://localhost:8081/swagger-ui.html`.

Diagrama



Prueba Parte 1 — Guion completo antes del refactor

```
1. POST Programa
POST /api/programas
{"codigo": "PROG-ING", "nombre": "Ingeniería de Software"}
→ HTTP 201 ✔  {"success": true, "message": "Programa creado", "data": {"id": 4, "codigo": "PROG-ING",
"nombre": "Ingeniería de Software"}}

2. POST Asignatura con programaId
POST /api/asignaturas
{"codigo": "ASIG-IS3", "nombre": "Ingeniería de Software III", "creditos": 3, "programaId": 4}
→ HTTP 201 ✔  {"success": true, "message": "Asignatura creada", "data": {"id": 11, "codigo": "ASIG-IS3",
"nombre": "Ingeniería de Software III", "creditos": 3, "programaId": 4, "programaNombre": "Ingeniería de
Software"}}

3. POST Grupo con cupoMax=2
POST /api/grupos
{"codigoGrupo": "G-IS3-01", "cupoMax": 2, "asignaturaId": 11}
→ HTTP 201 ✔  {"success": true, "message": "Grupo creado", "data": {"id": 21, "codigoGrupo": "G-IS3-01",
"cupoMax": 2, "asignaturaId": 11, "asignaturaNombre": "Ingeniería de Software III"}}

4. POST Estudiante
POST /api/estudiantes
{"nombre": "Ana Torres", "email": "ana.torres@uis.edu.co"}
→ HTTP 200 ✔  {"id": 51, "nombre": "Ana Torres", "email": "ana.torres@uis.edu.co"}

5. POST Matrícula 1 (OK)
POST /api/matriculas
{"estudianteId": 51, "grupoId": 21}
→ HTTP 201 ✔  {"success": true, "message": "Matrícula registrada", "data": {"id": 1, "estudianteId": 51,
"estudianteNombre": "Ana Torres", "grupoId": 21, "codigoGrupo": "G-IS3-01"}}

6. POST Matrícula 2 (OK)
POST /api/matriculas
{"estudianteId": 1, "grupoId": 21}
→ HTTP 201 ✔  {"success": true, "message": "Matrícula registrada", "data": {"id": 2, "estudianteId": 1,
"estudianteNombre": "Patricia Gutiérrez Escalante", "grupoId": 21, "codigoGrupo": "G-IS3-01"}}

7. POST Matrícula 3 al mismo grupo (falla por cupo)
POST /api/matriculas
{"estudianteId": 2, "grupoId": 21}
→ HTTP 409 ✔  {"error": "Conflict", "message": "El grupo no tiene cupo disponible", "status": 409}

8. POST Matrícula duplicada (falla por duplicado)
POST /api/matriculas
{"estudianteId": 51, "grupoId": 21}
→ HTTP 409 ✔  {"error": "Conflict", "message": "El estudiante ya está matriculado en ese grupo",
"status": 409}

9. POST Matrícula con estudiante inexistente (NotFound)
POST /api/matriculas
{"estudianteId": 9999, "grupoId": 21}
→ HTTP 404 ✔  {"error": "Not Found", "message": "Estudiante no encontrado: 9999", "status": 404}

10. POST Matrícula sin grupoId (validación)
POST /api/matriculas
{"estudianteId": 1}
→ HTTP 400 ✔  {"error": "Bad Request", "message": "Validación inválida", "fields": {"grupoId": "grupoId
es obligatorio"}, "status": 400}

Conteos tras el seeder + guion
programas: 4, asignaturas: 11, grupos: 21
```

RESULTADO: 10/10 pasos con el status esperado: 2 matrículas exitosas (pasos 5 y 6), 1 error por cupo (7) y 1 error por duplicado (8). Los pasos 9 y 10 prueban además el NotFound y la validación.

Parte 2 — Estructura por módulos

```
asignaturas.controller   AsignaturaController
asignaturas.dto          AsignaturaCreateDTO, AsignaturaDTO
```

```
asignaturas.handler      AsignaturaHandler
asignaturas.model        Asignatura
asignaturas.repository   AsignaturaRepository
asignaturas.service      AsignaturaServiceImpl, AsignaturaService
common.api               ApiResponse, ResponseBuilder
common.controller        GlobalExceptionHandler
common.exception         BusinessException, NotFoundException
config                   DataSeederAcademico, DataSeeder
(raíz)                   DemoAcademicoApplication
estudiantes.controller   EstudianteController
estudiantes.model        Estudiante
estudiantes.port         EstudianteQueryPort
estudiantes.repository   EstudianteRepository
estudiantes.service      EmailDuplicadoException, EstudianteNoEncontradoException, EstudianteService
grupos.controller        GrupoController
grupos.dto               GrupoCreateDTO, GrupoDTO
grupos.handler           GrupoHandler
grupos.model             Grupo
grupos.port              GrupoQueryPort
grupos.repository        GrupoRepository
grupos.service           GrupoServiceImpl, GrupoService
matriculas.controller    MatriculaController
matriculas.dto           MatriculaCreateDTO, MatriculaDTO
matriculas.handler       MatriculaHandler
matriculas.model         Matricula
matriculas.repository    MatriculaRepository
matriculas.service       MatriculaServiceImpl, MatriculaService
programas.controller     ProgramaController
programas.dto            ProgramaCreateDTO, ProgramaDTO, ProgramaUpdateDTO
programas.handler        ProgramaHandler
programas.model          Programa
programas.repository     ProgramaRepository
programas.service        ProgramaServiceImpl, ProgramaService
```

Parte 2 — MatriculaServiceImpl: de repositorios a Ports

```
-package co.edu.demoacademico.service;
+package co.edu.demoacademico.matriculas.service;
-import co.edu.demoacademico.exception.BusinessException;
-import co.edu.demoacademico.model.Estudiante;
-import co.edu.demoacademico.model.Grupo;
-import co.edu.demoacademico.model.Matricula;
-import co.edu.demoacademico.repository.EstudianteRepository;
-import co.edu.demoacademico.repository.GrupoRepository;
-import co.edu.demoacademico.repository.MatriculaRepository;
+import co.edu.demoacademico.common.exception.BusinessException;
+import co.edu.demoacademico.estudiantes.model.Estudiante;
+import co.edu.demoacademico.estudiantes.port.EstudianteQueryPort;
+import co.edu.demoacademico.grupos.model.Grupo;
+import co.edu.demoacademico.grupos.port.GrupoQueryPort;
+import co.edu.demoacademico.matriculas.model.Matricula;
+import co.edu.demoacademico.matriculas.repository.MatriculaRepository;
-    private final EstudianteRepository estudianteRepo;
-    private final GrupoRepository grupoRepo;
+    private final EstudianteQueryPort estudiantePort;
+    private final GrupoQueryPort grupoPort;
-    public MatriculaServiceImpl(EstudianteRepository estudianteRepo,
-                                GrupoRepository grupoRepo,
+    public MatriculaServiceImpl(EstudianteQueryPort estudiantePort,
+                                GrupoQueryPort grupoPort,
-        this.estudianteRepo = estudianteRepo;
-        this.grupoRepo = grupoRepo;
+        this.estudiantePort = estudiantePort;
+        this.grupoPort = grupoPort;
-        // 1) existencia (usa el NotFound del Lab 02 en los services/handlers en la Parte 2)
-        Estudiante estudiante = estudianteRepo.findById(estudianteId)
-                .orElseThrow(() -> new co.edu.demoacademico.exception.NotFoundException("Estudiante no
encontrado: " + estudianteId));
+        // existencia se valida por contrato público (y lanza NotFound desde el módulo dueño)
+        Estudiante estudiante = estudiantePort.obtenerPorId(estudianteId);
+        Grupo grupo = grupoPort.obtenerPorId(grupoId);
-        Grupo grupo = grupoRepo.findById(grupoId)
```

```
-                .orElseThrow(() -> new co.edu.demoacademico.exception.NotFoundException("Grupo no
encontrado: " + grupoId));
-        // 2) no duplicado
+        // no duplicado
-        // 3) cupo
+        // cupo
-        // 4) guardar
```

Verificación arquitectónica (B7)

```
grep -rn "import ...(estudiantes|grupos)" matriculas/
matriculas/model/Matricula.java:3:import co.edu.demoacademico.estudiantes.model.Estudiante;
matriculas/model/Matricula.java:4:import co.edu.demoacademico.grupos.model.Grupo;
matriculas/service/MatriculaServiceImpl.java:4:import co.edu.demoacademico.estudiantes.model.Estudiante;
matriculas/service/MatriculaServiceImpl.java:5:import
co.edu.demoacademico.estudiantes.port.EstudianteQueryPort;
matriculas/service/MatriculaServiceImpl.java:6:import co.edu.demoacademico.grupos.model.Grupo;
matriculas/service/MatriculaServiceImpl.java:7:import co.edu.demoacademico.grupos.port.GrupoQueryPort;
```

No hay imports a `estudiantes.repository` ni a `grupos.repository`; sí a `EstudianteQueryPort` y `GrupoQueryPort`. Los imports de `model` son las entidades que la matrícula referencia.

```
mvn clean package
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.077 s -- in
co.edu.demoacademico.ArquitecturaMatriculasTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.920 s -- in
co.edu.demoacademico.DemoAcademicoApplicationTests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Prueba Parte 2 — Mismo guion después del refactor (no regresión)

```
1. POST Programa
POST /api/programas
{"codigo": "PROG-ING", "nombre": "Ingeniería de Software"}
→ HTTP 201 ✔  {"success": true, "message": "Programa creado", "data": {"id": 4, "codigo": "PROG-ING",
"nombre": "Ingeniería de Software"}}

2. POST Asignatura con programaId
POST /api/asignaturas
{"codigo": "ASIG-IS3", "nombre": "Ingeniería de Software III", "creditos": 3, "programaId": 4}
→ HTTP 201 ✔  {"success": true, "message": "Asignatura creada", "data": {"id": 11, "codigo": "ASIG-IS3",
"nombre": "Ingeniería de Software III", "creditos": 3, "programaId": 4, "programaNombre": "Ingeniería de
Software"}}

3. POST Grupo con cupoMax=2
POST /api/grupos
{"codigoGrupo": "G-IS3-01", "cupoMax": 2, "asignaturaId": 11}
→ HTTP 201 ✔  {"success": true, "message": "Grupo creado", "data": {"id": 21, "codigoGrupo": "G-IS3-01",
"cupoMax": 2, "asignaturaId": 11, "asignaturaNombre": "Ingeniería de Software III"}}

4. POST Estudiante
POST /api/estudiantes
{"nombre": "Ana Torres", "email": "ana.torres@uis.edu.co"}
→ HTTP 200 ✔  {"id": 51, "nombre": "Ana Torres", "email": "ana.torres@uis.edu.co"}

5. POST Matrícula 1 (OK)
POST /api/matriculas
{"estudianteId": 51, "grupoId": 21}
→ HTTP 201 ✔  {"success": true, "message": "Matrícula registrada", "data": {"id": 1, "estudianteId": 51,
"estudianteNombre": "Ana Torres", "grupoId": 21, "codigoGrupo": "G-IS3-01"}}

6. POST Matrícula 2 (OK)
POST /api/matriculas
{"estudianteId": 1, "grupoId": 21}
→ HTTP 201 ✔  {"success": true, "message": "Matrícula registrada", "data": {"id": 2, "estudianteId": 1,
"estudianteNombre": "Víctor Venegas Aranda", "grupoId": 21, "codigoGrupo": "G-IS3-01"}}
```

```
7. POST Matrícula 3 al mismo grupo (falla por cupo)
POST /api/matriculas
{"estudianteId": 2, "grupoId": 21}
→ HTTP 409 ✔  {"error": "Conflict", "message": "El grupo no tiene cupo disponible", "status": 409}

8. POST Matrícula duplicada (falla por duplicado)
POST /api/matriculas
{"estudianteId": 51, "grupoId": 21}
→ HTTP 409 ✔  {"error": "Conflict", "message": "El estudiante ya está matriculado en ese grupo",
"status": 409}

9. POST Matrícula con estudiante inexistente (NotFound)
POST /api/matriculas
{"estudianteId": 9999, "grupoId": 21}
→ HTTP 404 ✔  {"error": "Not Found", "message": "Estudiante no encontrado: 9999", "status": 404}

10. POST Matrícula sin grupoId (validación)
POST /api/matriculas
{"estudianteId": 1}
→ HTTP 400 ✔  {"error": "Bad Request", "message": "Validación inválida", "fields": {"grupoId": "grupoId
es obligatorio"}, "status": 400}

Conteos tras el seeder + guion
programas: 4, asignaturas: 11, grupos: 21
```

RESULTADO: 10/10 pasos con el status esperado. Las respuestas son idénticas a las de la Parte 1 (comparadas con `diff`, ignorando la fecha de registro y el nombre aleatorio del estudiante 1).

